Week 8: ELT Overview

Keeyong Han

Table Of Contents

1. Recap of Data Pipeline
2. Quiz #2 Review
3. ELT Overview
4. CTAS based ELT as a DAG in Airflow
5. Break
6. Dashboard Overview
7. Dashboard Integration: Superset
8. Demo & Homework #7
9. Lab #2 Details
10. Group Project

Recap of the 7th Week

Let’s say if you do an **daily** update of a stock price

- Everyday, get the price of the previous day
- Naive approach is to compute dates of data to fetch yourself
  - Everyday, you need to know yesterday’s date and today’s date. Run the following code

```
from datetime import datetime, timedelta

today = datetime.today().strftime('%Y-%m-%d')   # Get today's date
yesterday = (datetime.today() - timedelta(days=1)).strftime('%Y-%m-%d') # Get yesterday's date

data = yf.download("APPL", start=yesterday, end=today)
```

Is that enough?

What if we learned this run didn't go well later?
You have to modify the code and rerun

- What's the problem of the approach?
  - What if you have to go back and rerun any past runs? (Failed DAG Run)
- Let's say there is a pipeline running daily at 2:30AM
  - When it runs on 01/02, the data to fetch is from 01/01
  - …
  - When it runs on 03/03, the data to fetch is from 03/02
  - When it runs on 03/04, the data to fetch is from 03/03

DAG & DAG Run

Backfill will be a problem

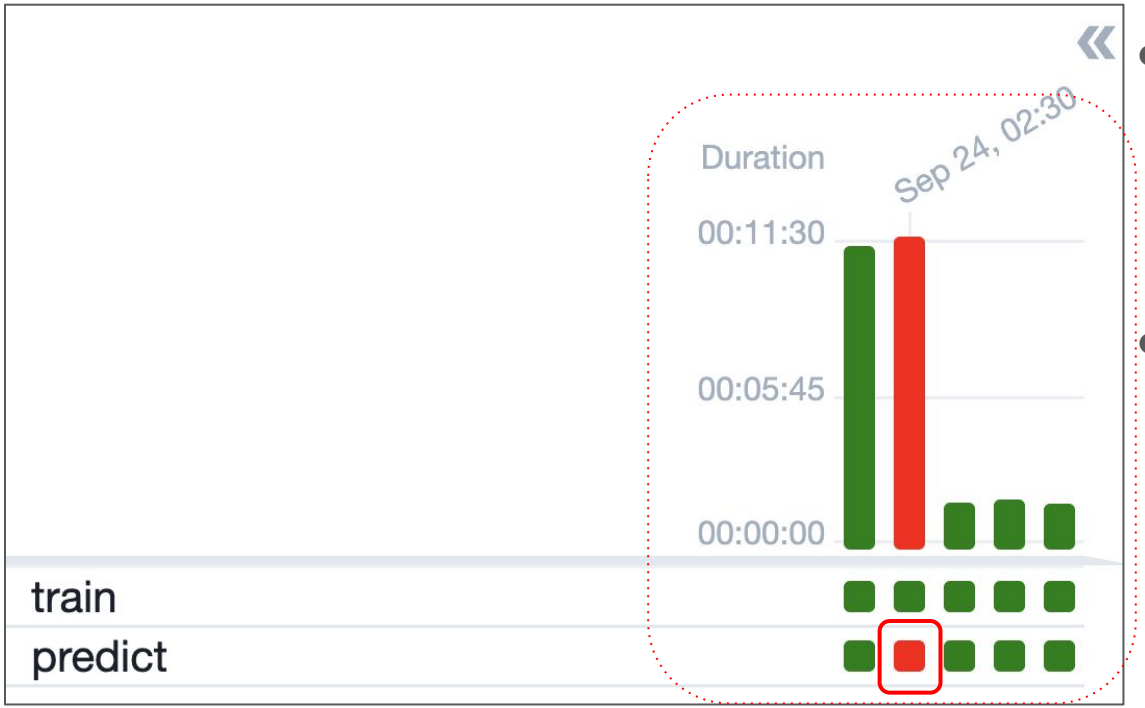
- the logic is built to get the dates **at the time of data pipeline's run (DAG Run)**
  - We need to record the dates of each data pipeline's run somewhere
- How to implement this mechanism?

How Airflow can be helpful

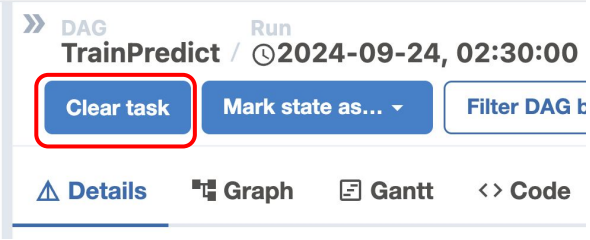
- For each DAG run, Airflow records its run statuses and associated timestamps
  - The **meta database** is used to store this information
  - There is a table called “dag_run” in the database
- Let’s say there is a DAG running daily at 2:30AM
  - When it runs on 03/03, the data to fetch is from 03/02
  - When it runs on 03/04, the data to fetch is from 03/03
  - …

Airflow has a variable called “**logical_date**” which associates each DAG run with the data date to fetch

Airflow's Approach



- You don't compute the date or the hour of data to fetch yourself but use whatever Airflow tells you ("logical_date")
- If there is any failed run, it will appear as red in the Web UI and you can rerun it



The results are stored in dag_run table

How to get the logical_date in Airflow

from airflow.operators.python import **get_current_context**

```
def get_logical_date():
    # Get the current context
    context = get_current_context()
    # Get the logical_date as a string from the context
    logical_date = str(context['logical_date'])
    # Return the first 10 characters which is “YYYY-MM-DD”
    return logical_date[:10]
```

```
context['ds']
context['logical_date']
…
```

How Airflow can get you: **start_date**

- Airflow assumes that a DAG is doing incremental update
- DAG's **start_date** refers to the date of data to copy for the first time
- If it is '2025-01-01' for a daily DAG, then the first time the DAG runs is:
  - **2025-01-02**

Quiz #1 & #2

If a DAG has schedule of “30 2 * * *” and start_date of “2025-03-01”

1. When would it run for the first time?
   a. 2025-03-02
2. If I enable this DAG on 03/06, what will happen to the missing runs (from 2025-03-01 to 2023-03-05)?
   a. **Catchup** will determine!
   b. Its default value is **True**. If it is True, it will try to catch up any missing runs literally

Quiz #3

3. When a DAG has schedule of “30 2 * * *” and it runs on 03/05, what will be the values of the following variables?

context['logical_date']	2025-03-04 02:30:00
context['ds']	2025-03-04
…	

,

Quiz #2 Review

ELT Overview

Data Flow

Site traffic &
3rd party data

1

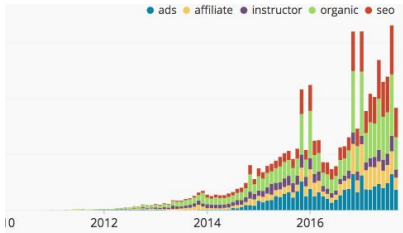
ELT

ETL

Data warehouse

Data Team

2



Business
Insights

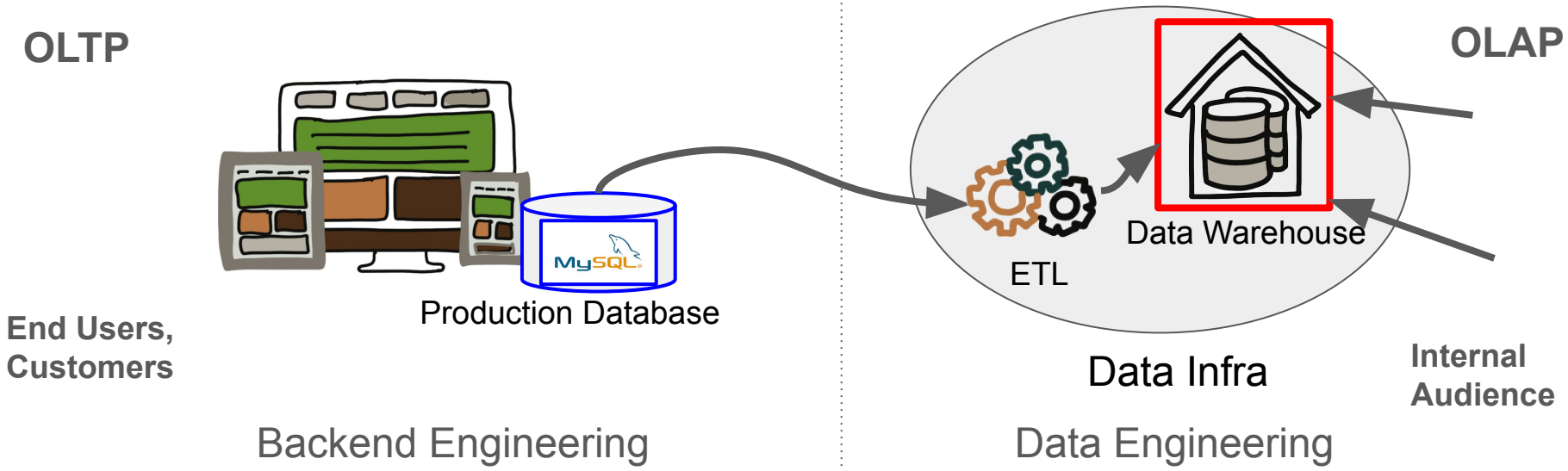
3

Site Services

Scalable Product Improvement

Data Warehouse: A separate database only for Data Stuffs

- Building a data warehouse is the first step to becoming a true data org
- Data Warehouse should be separate from the production database.
  - Dedicated database for leveraging data vs. production database

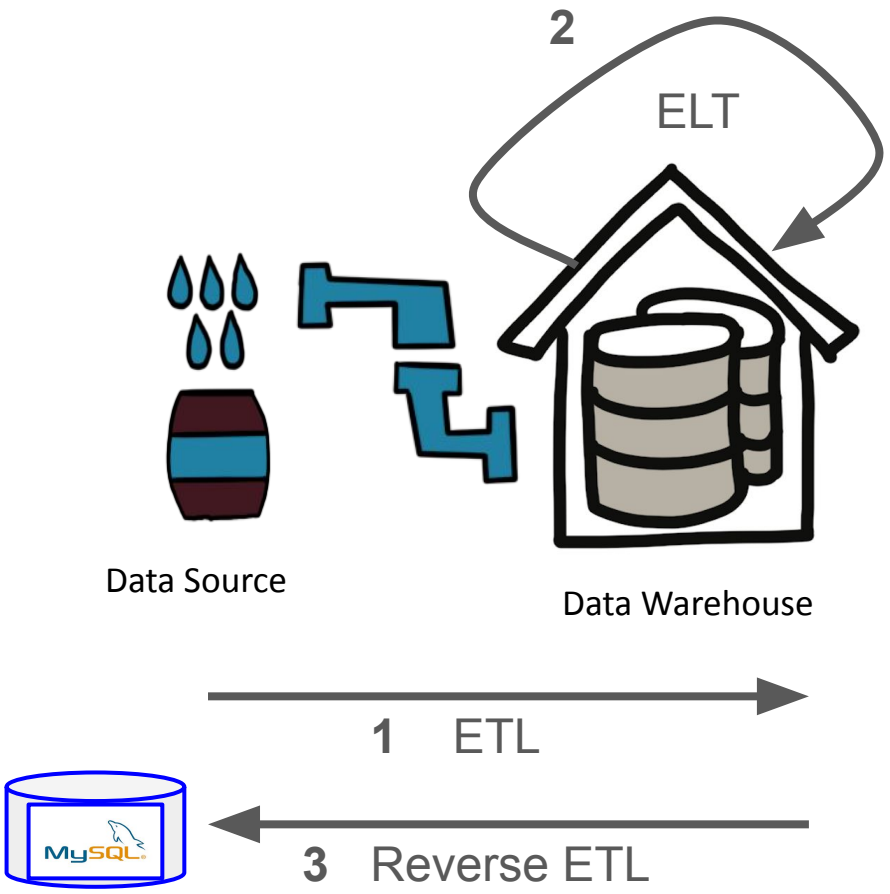


Difference between OLTP and OLAP (both are databases)

- OLTP: Production Database
  - Used to store data needed for operating the service/product
    - processing speed is more important
  - Managed by **Backend** engineering
  - MySQL, PostgreSQL, Oracle, …
    - A single server solution with limited capacity (Scale Up)
- OLAP: Data Warehouse
  - Used to store data needed for data analysis and other data related work
    - size of data to handle is more critical
    - In other words, scalability is the key (and cost-effectiveness)
  - Managed by **Data** engineering
  - Snowflake, BigQuery, Redshift, Hive/Presto, ClickHouse, …
    - A group of servers behind the scene (Scale Up and Out)

Types of Data Pipeline

1. ETL
2. ELT (“T”)
3. Reverse ETL



ETL vs. ELT (Both are Pipelines not databases)

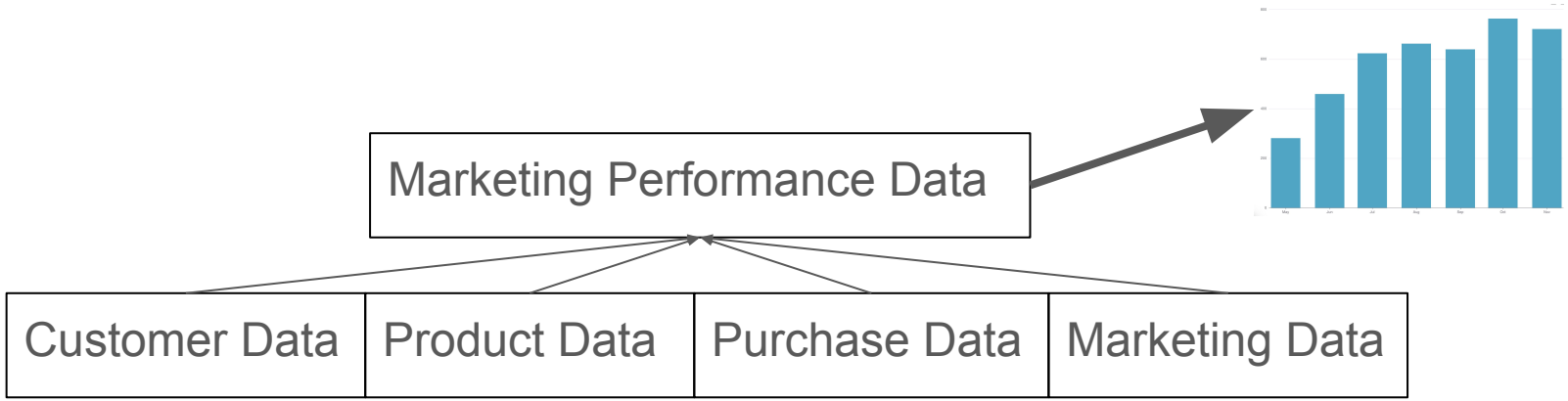
ETL ≠ OLTP & ELT ≠ OLAP

- ETL
  - The process of bringing data from outside the data system to inside
    - Data system can be data warehouse or data lake
  - Done by data engineers
  - One of the key data sources is the production database (OLTP)
- ELT
  - The process of manipulating data inside the data system to create new data (usually more abstracted and cleaned)
  - The most popular tool is dbt
  - Done by data analysts or analytics engineers
  - Input to ELT can be in the data lake

Airflow is used to schedule and/or develop pipelines

What is the future of ELT?

- We build ETLs so that we can get data insights in some ways
- We run ELTs to get the insights by creating abstract layers
- Data quality validation becomes extremely important



Increasing Importance of Data Quality

- Input/output checks (in particular for ELTs)
- More diverse quality inspections
- Lineage checks
- Understanding data change history

This is why **dbt** is needed

- Maintaining data quality is more important than ever
  - Reducing cost/effort and increasing productivity
  - Main reasons to have **data governance**

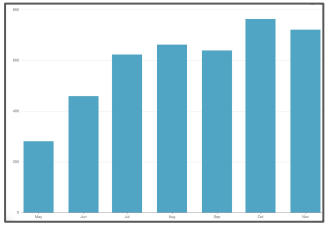
CTAS based ELT as a DAG in Airflow

Let’s build a MAU dashboard

- We will use CTAS
- Input tables:
  - raw.user_session_channel
    - userId, sessionId, channel
  - raw.session_timestamp
    - sessionId, ts
- Output table
  - analytics.session_summary

userId	sessionId	**ts**	channel
100	abcdedff	2024-10-01	google
101	bcdfegh	2024-10-01	facebook
102	cdegjhig	2024-10-02	direct

- We will use Superset
- Input table:
  - analytics.session_summary
- Output Chart/Dashboard
  - MAU chart



The Simplest ELT Implementation

```
CREATE OR REPLACE TABLE dev.analytics.session_summary AS
SELECT u.*, s.ts
FROM dev.raw.user_session_channel u
JOIN dev.raw.data.session_timestamp s ON u.sessionId=s.sessionId;
```

CTAS

```
SELECT sessionId, COUNT(1) AS cnt
FROM dev.analytics.session_summary
GROUP BY 1
ORDER BY 2 DESC
LIMIT 1
```

Primary Key Uniqueness Check
cnt should be 1

The Simplest ELT Implementation

- (Ideal) Overall flow
  - Start transaction
  - Drop the table
  - Create the table using CTAS
  - Perform various validation such as primary key uniqueness check
    - If any fails, throw an exception
  - Commit the transaction

Snowflake does not treat DDL statements as part of a transaction; each DDL command commits automatically

The Simplest ELT Implementation

- (Snowflake) Overall flow
  - Create a temp table using CTAS
  - Perform various validations against the temp table such as primary key uniqueness check
    - If it fails, throw an exception
  - Create the final table if it doesn't exist
  - Swap the temp table and the final table
    - This is a DDL so it won't honor the transaction and will be committed immediately

But what if the final table doesn't exist?
It will cause an error so we need an extra step of creating the final table before swapping

The Simplest ELT Implementation

- GitHub Code
- Demo

Break

Dashboard Overview (Superset In Particular)

What is Dashboard?

- Also referred to as a BI (Business Intelligence) tool
- The most common use is to visualize metrics
  - KPI (Key Performance Indicators) such as revenue, paid users, and so on
- Enables decision-makers to make more scientific decisions
  - Data-Driven Decision
  - Data-Informed Decision
- Things to watch out
  - Death by a thousand dashboards
  - Next metrics fallacy

Types of Dashboards

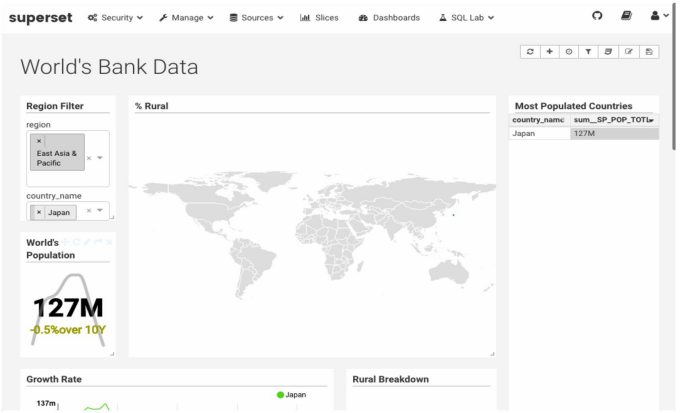
- Excel, Google Spreadsheet: The most widely used tools
- Looker (Google)
- Tableau (Salesforce)
- Power BI (Microsoft)
- Apache Superset, ReDash (Opensource)
- Mode Analytics (Startup)
- Google Studio, AWS Quicksight
- Python: More suitable for EDA (Exploratory Data Analysis)

What Dashboard tool to Choose?

- Looker and Tableau are among the most commonly used tools today.
  - Both tools require some time to learn initially.
  - Tableau is more affordable and transparent in terms of pricing, and it even offers a free version, making it accessible for learning
- The key point is creating self-service dashboards
  - If 60-70% of questions can be answered through self-service dashboards, it's a major success
  - In this aspect, Looker may be a better choice, but it comes with a significantly higher price tag and the quality is somewhat deteriorating

Superset: An Open Source from Airbnb

- The author of Airflow is the creator (Maxim)
  - https://github.com/apache/superset
  - Airbnb, Dropbox and so on are using Superset
- Maxim founded a company to commercialize
  - https://preset.io/

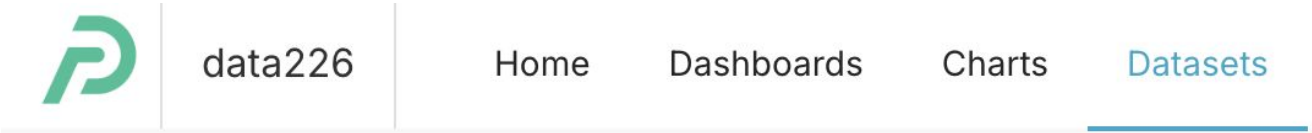


What is Superset?

- Supports various forms of visualization and an easy-to-use interface
- Supports dashboard sharing
- Provides enterprise-level security and access control features
- Supports various databases (OLTP and OLAP)
- Enables real-time data visualization through integration with Druid.io
- Offers good scalability thanks to API and plugin architecture

Superset Terminology

- Database/Dataset
  - A Database == Ex) Snowflake, Redshift, …
  - A Dataset == A Table
- Dashboard/Chart
  - A Dashboard consists of 1+ Chart



Superset Options

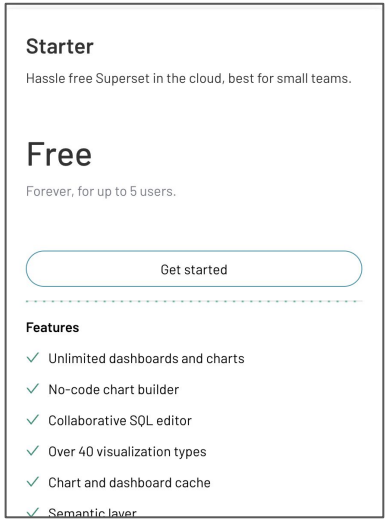
- What to use is your call
- Cloud version: Free Starter works
  - With the free version limitation gone, we will demo this version
  - https://preset.io/
- Docker version
  - You can use this one as well
- Tableau Public is another way to go
  - But you can’t connect a database (can only upload a file)
  - Student License has changed in the last few months and it no longer supports database connection and became same as Tableau Public.

Demo & Homework #6

Dashboard Integration: Preset

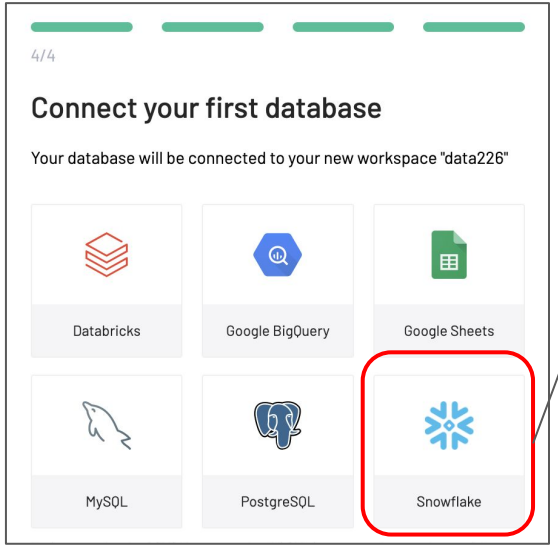
Sign up **Preset**

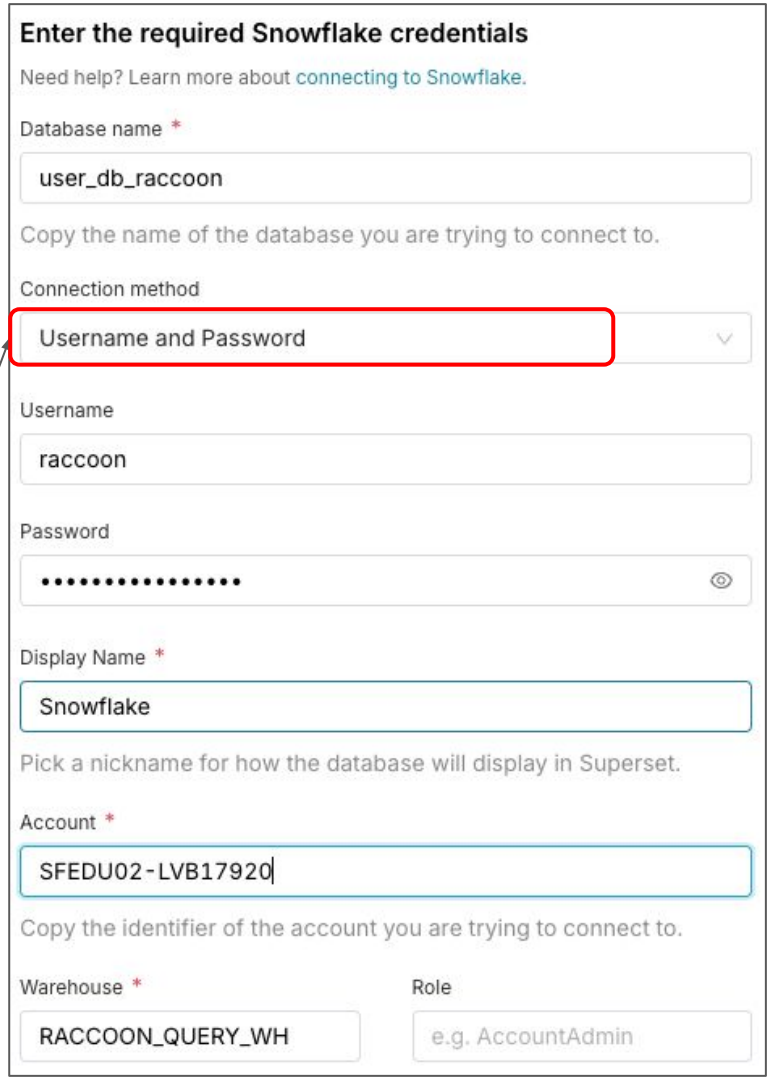
1. Sign up at https://preset.io/
   a. **Starter Free Plan** is fine
2. Create a workspace



Set up Snowflake Connection

3. Configure your Snowflake database setting

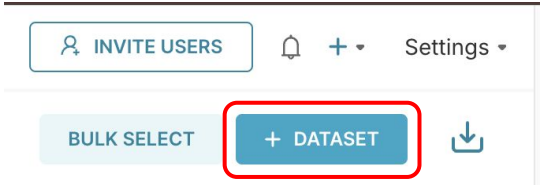


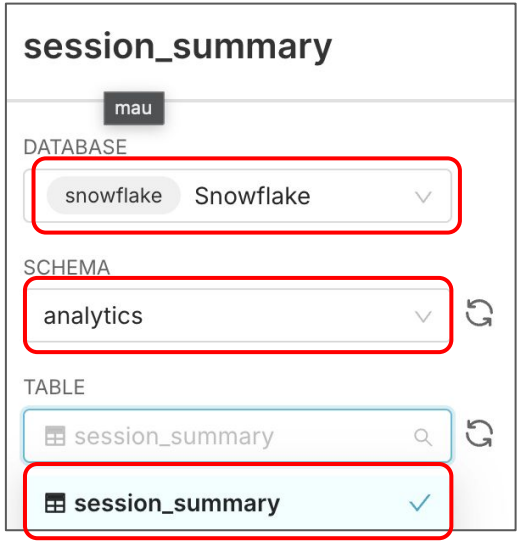


This connection can take a while!

Load analytics.session_summary table as a dataset

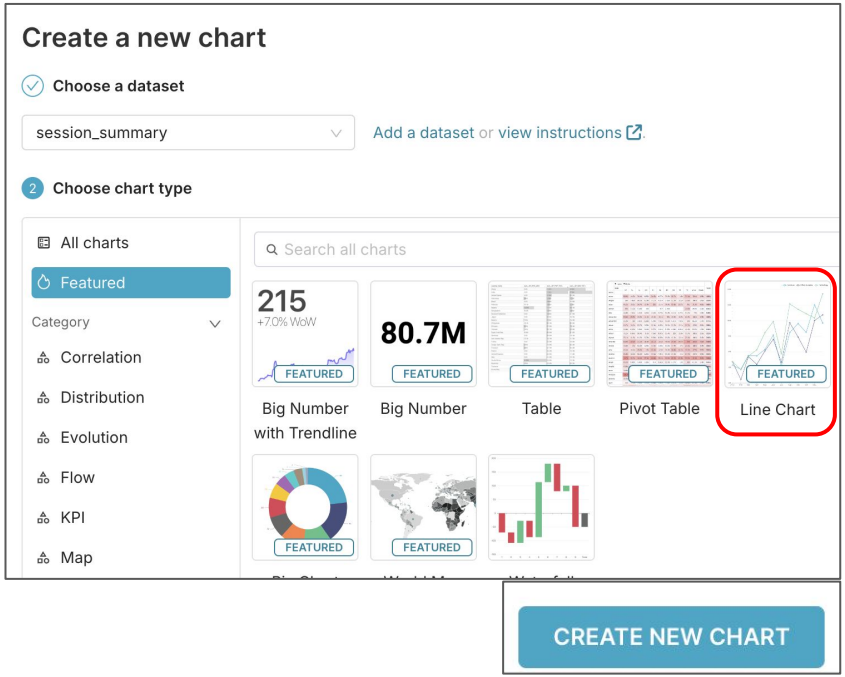
4. Select “Datasets” and then click “+ DATASET” on the top right corner
5. In the left box select DATABASE, SCHEMA, and TABLE accordingly
6. Click “CREATE DATASET AND CREATE CHART” on the bottom right corner





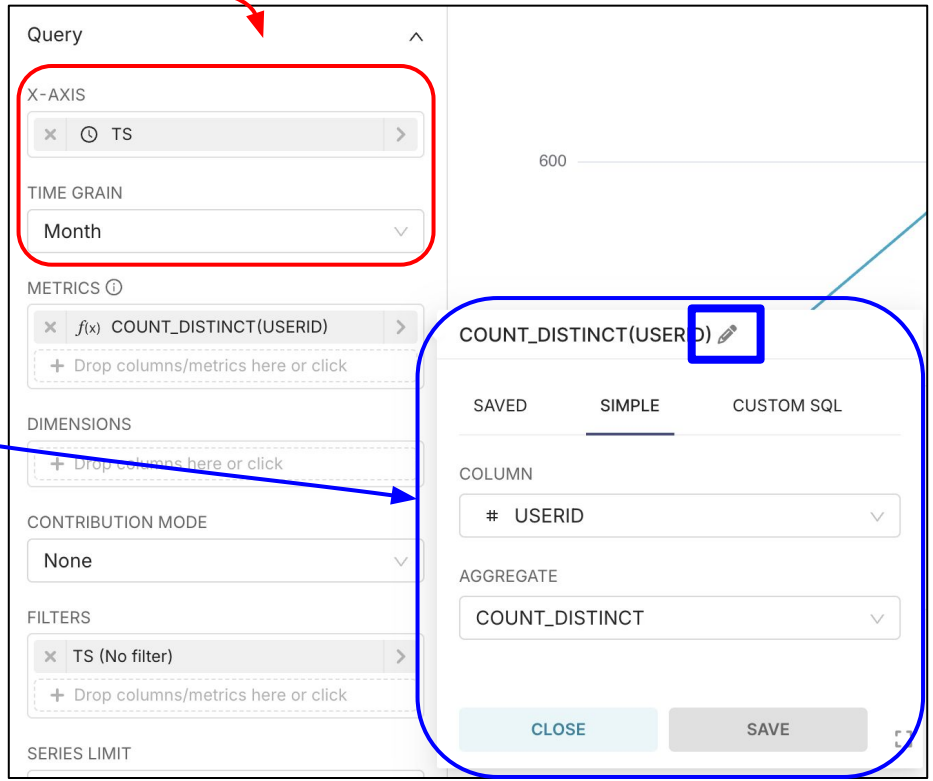
Create a Line Chart (#1)

7. In the “Create a new chart”, select Line Chart -> “CREATE NEW CHART”



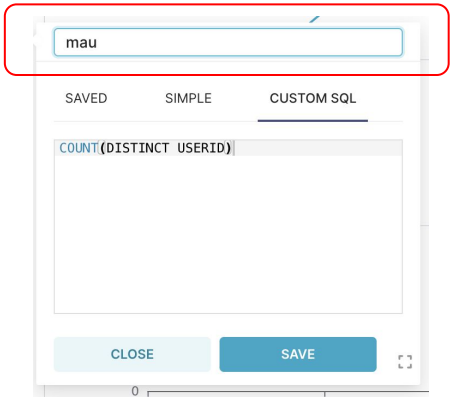
Create a Line Chart (#2)

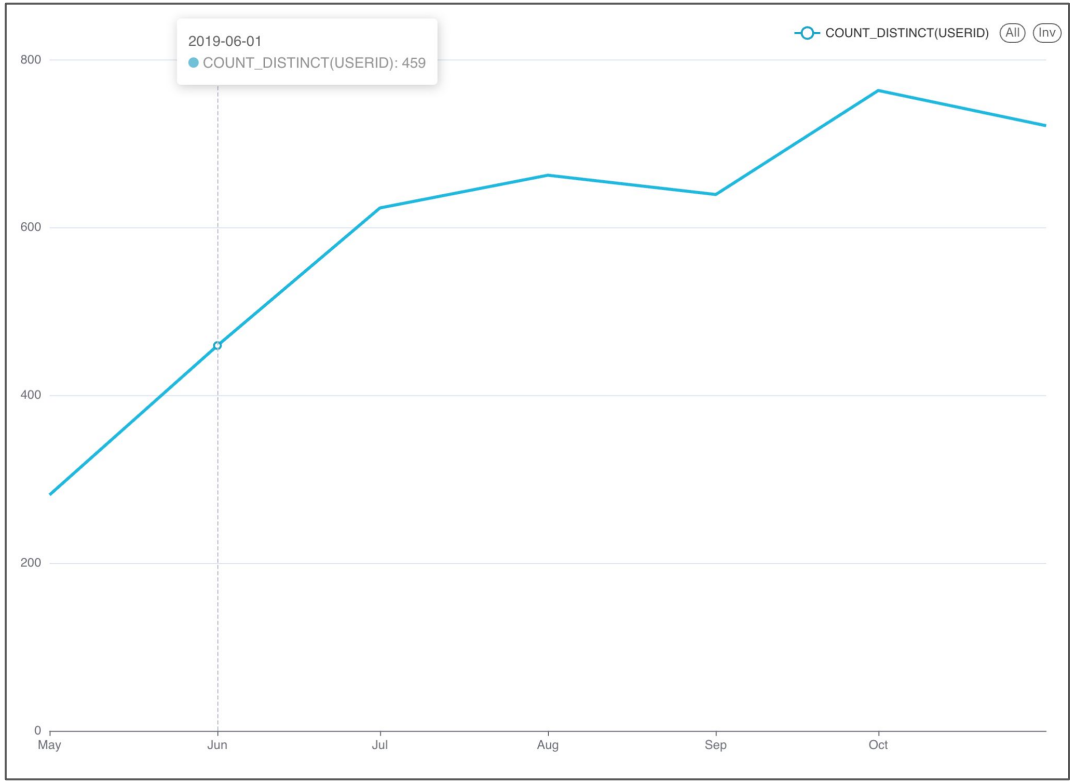
8. Fill in **X-Axis** and **Metrics** as indicated here
9. Click “CREATE CHART”



Final Look of the Chart

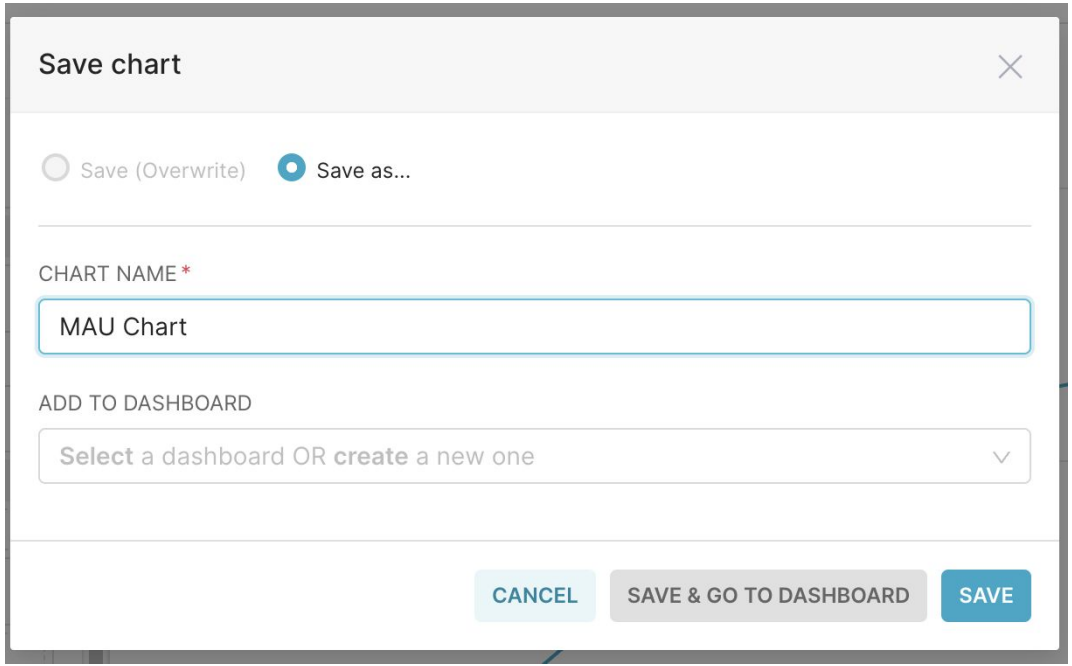
- See if you can do the following
  - Change the chart type to Bar
  - Rename COUNT_DISTINCT(USERID) to “mau”





Save the Chart

- You can save it as a dashboard as well
- Dashboard = 1+ Charts with 0+ Filters



Homework #6 (10 points)

Create a WAU (Weekly Active User) chart from any BI tool. Total 10 points

1. Import two tables in your Snowflake as an ETL DAG in your Airflow (+3pt)
    a. user_session_channel and session_timestamp (under raw schema or equivalent)
    b. Capture the screenshot of this DAG's detailed page from the Web UI (#1)
2. Create a ELT DAG in your Airflow to create a JOINED table of the two (+3pt)
    a. session_summary (under analytics)
    b. **Extra point**: add one more condition to check **duplicate** records (+1pt)
    c. Capture the screenshot of this DAG's detailed page from the Web UI (#2)
3. Set up your Preset account (+2pt)
    a. This includes setting up Snowflake connection and import session_summary from step 2
    b. Capture the screenshot of your Datasets (#3)
4. Create your WAU chart (+2pt)
    a. Make sure you rename the metrics field to **WAU**
    b. Capture the screenshot of the chart (#4)

All the codes from step 1 and 2 need to be in your Github repository and the repo needs to be submitted

Screenshot #1

DAG: SessionToSnowflake

Schedule: 15 1 * * *

Next Run ID:

10/14/2024 06:54:12 PM

All Run Types

All Run States

Clear Filters

Press shift + / for Shortcuts

deferred failed queued removed restarting running scheduled shutdown skipped success up_for_reschedule

Duration

00:00:04

00:00:02

00:00:00

set_stage

load

DAG SessionToSnowflake / Run 2024-10-14, 01:15:00 UTC

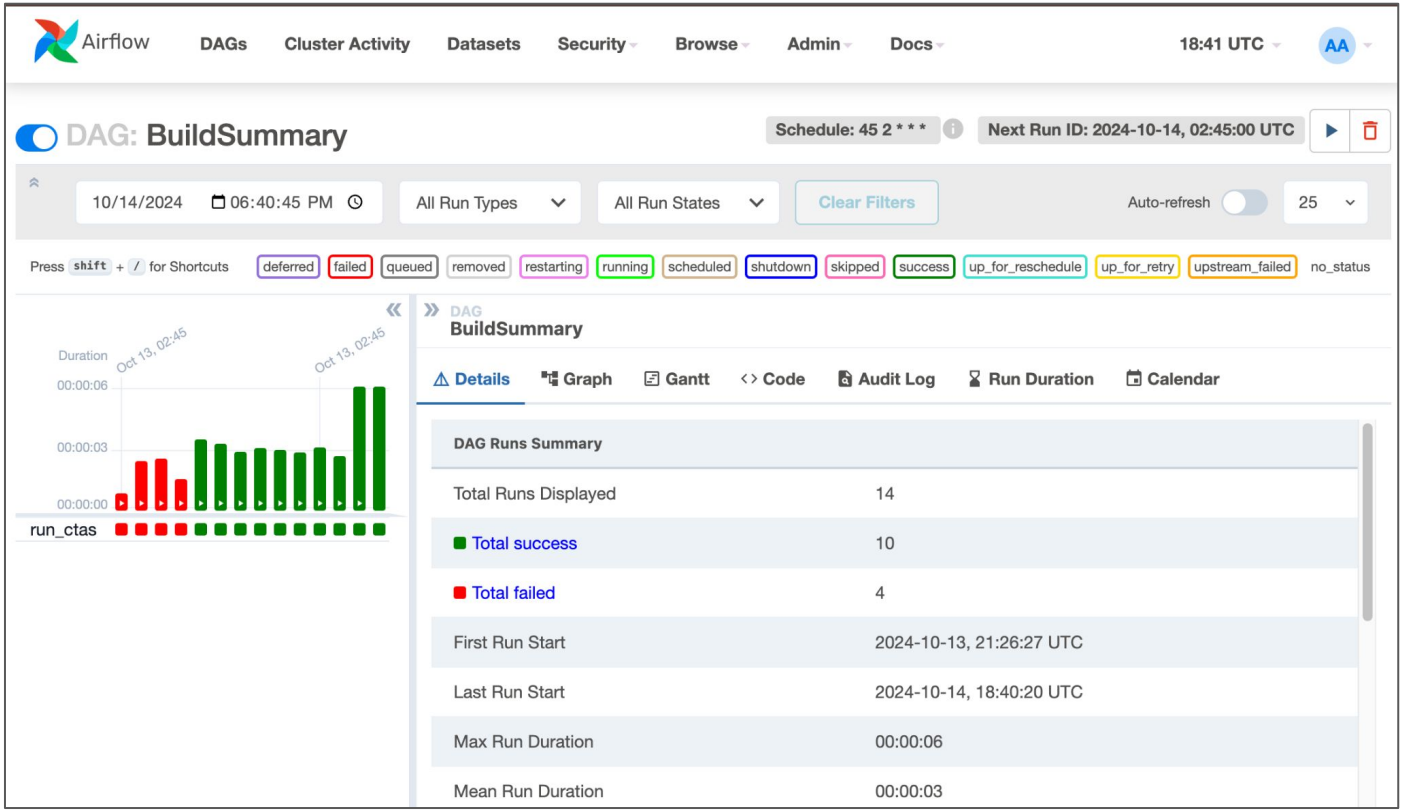
Details Graph Gantt Code Audit Log

DAG Run Notes:

Dag Run Details

Status	success
Run ID	scheduled__2024-10-13T01:15:00+00:00
Run type	scheduled
Run duration	00:00:04
Last scheduling decision	2024-10-14, 19:24:58 UTC

Screenshot #2



Screenshot #3

Superset Dashboards Charts Datasets SQL + Settings

Datasets

BULK SELECT + DATASET

NAME: Type a value
TYPE: Select or type a value
DATABASE: Select or type a value
SCHEMA: Select or type a value
OWNER: Select or type a value
CERTIFIED: Select or type a value
MODIFIED BY: Select or type a value

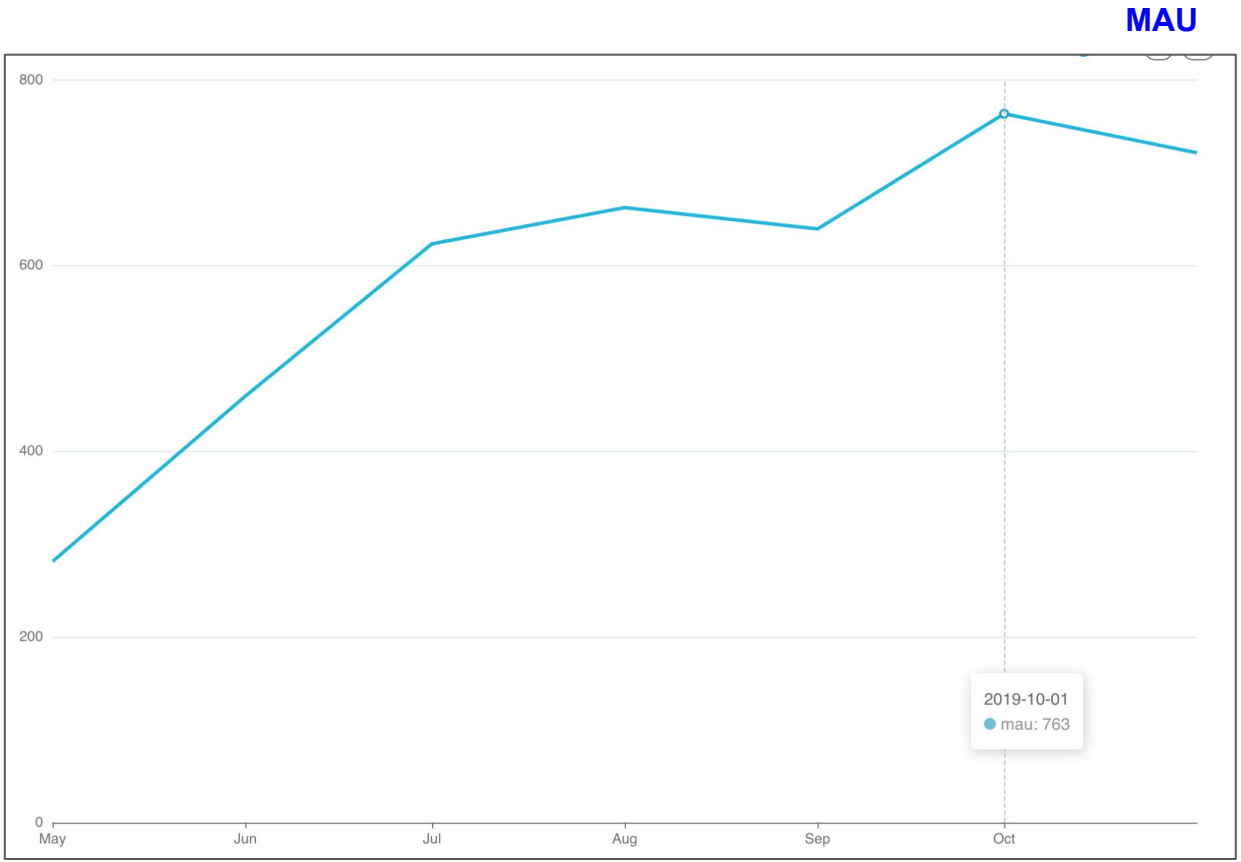
Name	Type	Database	Schema	Owners	Last modified	Actions
session_summary	Physical	Snowflake	analytics	AU	10 hours ago	

« 1 »

1-1 of 1

Screenshot #4

- This is an example (MAU)
- What you build is WAU not MAU
- The metric name should be WAU



Other Announcements

Group Project Proposal

- First proposal is due next week (10/21)

Final Exam Date

- Section 24 (3PM): 1:00 - 3:00 PM Dec 11
- Section 23 (6PM): 5:30 - 7:30 PM Dec 16

Lab #2 Overview (50 pts)

- Due date: 11/25 (week 14)
- Lab team formation will remain the same by default
- The goal is to build an ELT system with a dashboard
  - The dashboard of your choice can be one of Preset, Superset Docker and Tableau
  - What datasets to use
    i. This can be an extension of what you built in Lab #1
    ii. This can be something new focusing more on any historical data and insight from it

Lab #2 Details

Lab #2 Requirements

1. Populate raw data tables using Airflow
    a. Reuse your work from Lab 1 if applicable
    b. For new datasets, implement an ETL job in Airflow
    c. Capture screenshots accordingly
2. Create abstract table(s) using dbt
    a. Schedule dbt job(s) from Airflow
    b. Capture screenshots accordingly
3. Visualize insights using Preset or Superset or Tableau
    a. Base visualizations on the table(s) created in step 2
    b. Capture screenshots accordingly
4. Prepare the report in IEEE format
    a. Single-column format is acceptable, provided that it includes the same sections

All codes (including ETLs and dbt ELT) need to be in Github repository

Lab #2 Idea if you stick to the stock price project

1. Transformations in dbt
   a. Perform specific informative transformations (e.g., Moving Averages, RSI) in dbt.
   b. Once transformations are built, ensure the dbt models are run as part of an Airflow DAG.
2. Visualize Data in a BI tool (Superset or Preset or Tableau)
   a. Use a BI tool to visualize key metrics such as moving averages, RSI, price momentum, and so on you built from the step 1

We will learn about dbt after midterm

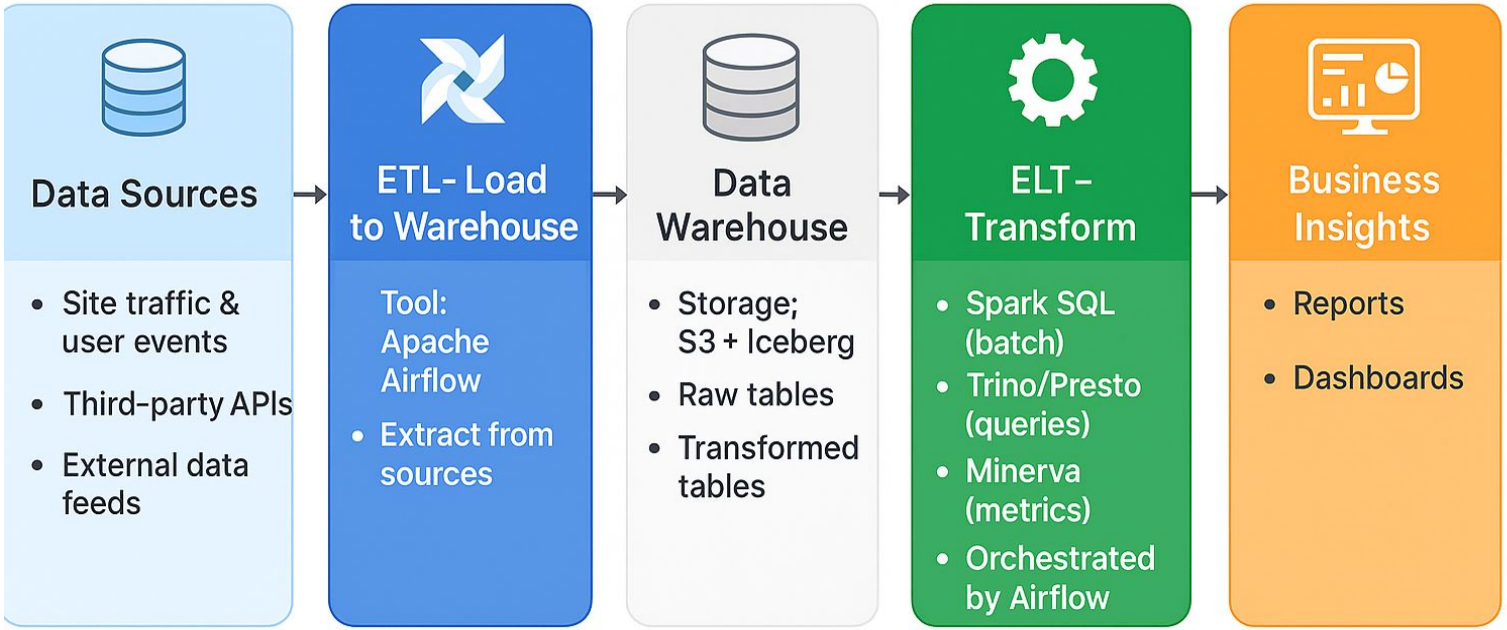
Details of Midterms (Lockdown Browser)

- Covers week 1 to week 8
- Part A:
  - 45 minutes & 30 questions (60 pts)
  - Multi-choice questions
- Part B:
  - 75 minutes & 11 questions (85 pts)
  - SQL queries: https://leetcode.com/studyplan/top-sql-50/
    - 2 questions from Easy SELECT will be modified (10 pts each)
    - 1 question from Easy BASIC JOIN will be modified (15 pts)
  - Concept assessment
    - Short-answer questions:
      - SQL: 2 questions (5 pts each)
      - Airflow: 4 questions (5 pts each)
    - Long-answer questions:
      - Data Pipelines: 2 questions (10 pts each)

Next Week

Midterm Exam

3PM: Professor Vishnu Pendyala

6PM: Professor Simon Shim

The Modern Data Stack in Production: How Airbnb Built It